

Embedded Systems II

EEE3096/5S



FINAL EXAM

16 November 2021

MEMO!!!!

2 hours

*Examination Prepared by:
Simon Winberg and Jane Wyngaard*

NB: THIS IS THE MEMO!!!!

REGULATIONS

Dear Students,

Please note this is a **closed-book exam**. Provide your answers to the questions in the space provided. An additional page is provided after the last question page. You are recommended to detach the cheat sheets that are on the last pages of the exam paper. The exam is printed double sided. You are recommended to first scan through the questions quickly before starting, so that you can plan your strategy for answering the questions. If you are caught cheating, you will be referred to University Court for expulsion procedures. Make sure to **put** your **student name and student number** above. If you need any additional pages, are going to submit your exam pad provided as rough work, then please indicated at least your student number on the script together with the **course code EEE3096S or EEE3095S and a title ES2 Final Exam**.

DO NOT TURN OVER UNTIL YOU ARE TOLD TO

Exam Structure

Marked out of 100 marks. 120 minutes.

RULES

- NB:**
- You must write your name and student number on each answer book.
 - Write the question numbers attempted on the cover of each book.
 - Start **each section on a new page**.
 - Make sure that you cross out material you do not want marked. Your first attempt at any question will be marked if two answers are found.
 - Use a part of your script to plan the facts for your written replies to questions, so that you produce carefully constructed responses.
 - Answer all questions, and note that the time for each question relates to the marks allocated (which implies there are a few minutes to allow you to think and shuffle papers etc).



Structure of exam:

Multiple Choice Short Answers Design Questions to test your mettle

SECTION A:

MULTIPLE CHOICE AND SOME EASYPEASY QUESTIONS [5 x 4 20 marks]

Circle the letter a - d to select your choice of answer (select only one answer option for each)

Question A.1. [4 marks]

In CPU pipelined to execute in stages, various potential hazards must be handled by the hardware. Executing the following assembly instructions will cause which type of hazard.

In a standard ARM CPU pipeline, execution follows which of the following stages (select one option)

- a) Decode, Execute, Memory Access, Fetch, Writeback
- b) Memory Access, Decode, Execute, Fetch, Writeback
- c) Memory Access, Fetch, Execute, Decode, Writeback
- d) Fetch, Decode, Execute, Memory Access, Writeback ← ANSWER

Question A.2. [4 marks]

There are various classifications for real-time systems. Which one of the following options best defines the characteristics of a periodic real-time system?

- a) A system with a set of tasks that executes repeatedly at a specific period. ← ANSWER
- b) A system that meets (at least one) specific hard real-time deadline that repeats at a specific interval.
- c) The term refers to a real-time system that uses polling (as opposed to e.g. interrupts).
- d) It is a system for which any one of its operations has to complete within a specific time bound.

Question A.3. [4 marks]

Operating systems are responsible for and provide:

- a) Process scheduling, and control of process access to hardware resources ← ANSWER
- b) Hardware management services such as mounting storage devices, controlling RAM access by different programs, and ensuring cache coherency is maintained.
- c) Process scheduling, I/O, and instruction pipelining services
- d) The system call memory table, boot loader storage location, and interrupt handling.

Question A.4. [4 marks]

Considering that the assembly function below is compatible with the C calling convention and compiler optimizations in use, what would be returned by func(133,1025) if called in a C function?

@ func(unsigned int, unsigned int):

```
add    r0, r1, r0
and    r0, r0, #127
bx     lr
```

- a) 0
- b) 6 ← ANSWER
- c) 8

NB: THIS IS THE MEMO!

d) 1158

Question A.5. [4 marks]

Consider that we want to count the number of bits that are clear (i.e., 0) in the input register x . The starting point to the module has been provided below. You need to select which option below is the right answer for which of these code options you can replace the '*put code here*' line to achieve the objective.

```
module numbits ( x, n );
    // define the ports
    input  [4:0] x;
    output [2:0] n;
    // define the registers
    reg n;

    // put code here!

endmodule
```

Select the right code option below:

a)

```
always @(x)
begin
    n = (x&1) + (x&2) + (x&4) + (x&8) + (x&16);
end
```

b) ← ANSWER

```
always @(x)
begin
    n = 5;
    if (x&1)  n = n - 1;
    else if (x&2)  n = n - 1;
    else if (x&4)  n = n - 1;
    else if (x&8)  n = n - 1;
    else if (x&16) n = n - 1;
end
```

c)

```
always @(x)
begin
    n = x[3]+x[2]+x[1]+x[0];
end
```

d)

```
always @(x)
begin
    n = x[0]^x[1]^x[2]^x[3];
end
```

SECTION B:

SHORT ANSWERS – NOT QUITE SO EASY QUESTIONS [23? marks]

Answer the following questions in the space provided below the question.

Question B1 [8 marks]

B1.1. There are many ARM processors that provide the facility of executing Thumb instructions. For example the ARM91TDMI is one such example. Briefly explain what a Thumb instruction is and how it may be of benefit when thinking of choosing a processor to use in an embedded system.

ANSWER: Expect students to discuss Thumb being able to fit two 16-bit instructions in one 32-bit word, providing savings in code space as well as time to load instructions from (non-cached) memory. This question is for 3 marks, so one mark can be for how logical the response is and connects with embedded systems needs, such as lower power.

[3 marks]

B1.2. Name another processor design extension that ARM processors may support (the name of the core used in the above subquestion gives some hints but you need to describe the extension not just its letter).

[2 marks]

ANSWER: Various ones like:

E : DSP instruction extensions

J : Jazelle, Java bytecode extensions

S : Softcore core available as synthesizable VHDL or Verilog code

T : Thumb 16 bit operations supported

B1.3. Answer the following true/false questions by circling True/False below.

Answer options:

ANSWER: MARKED IN RED

- The ARM processors can only run programs compiled in C True False
- RISC processors generally have less complex instructions than CISC processors True False
- When calling functions in C using the standard ABI for GCC you can only use a maximum of 7 parameters (as only 7 registers are permitted for function calls) True False

[3 marks]

Question B2 [7 marks]

B2.1. Why are IPCs necessary? Give 2 examples of IPCs and where they would be used.

[3 marks]

ANSWER: Because the OS ensures different processes cannot interfere with each other's memory allocations. So some other means is required to enable 1 process to pass information to another. [2]

Eg: Files, Pipes, Signals, Sockets, SHared memory, System Calls [0.5 each]

B2.2. Explain what is meant by the concepts of 'spatial locality' and 'temporal locality', emphasising how they differ (2 marks). Briefly explain why, if at all, these concepts lead to the usefulness of cache memory in computer system designs, instead of processors just using external memory that is cheaper and has lots of space available.

[4 marks]

ANSWER:

Temporal locality: The temporal locality principle states that if a data location is referenced then there is a high probability that it will be accessed again **soon**. [1]

Spatial locality: The spatial locality principle states that if a data location is referenced, then there is a high probability that data locations with **nearby** addresses will be accessed soon. [1]

Cache hierarchies exploit the above to **place the data most likely to next be required by the processor in the fastest memory locations closest to the processor**. Spatial and temporal locality mean the memory manager is able to make **reliable predictions regarding what the next data to be needed** will be. [2 + 1 mark for elegance/logic of answer]

Marking notes:

- A portion of students referring to the hierarchy in reverse (saying higher level cache is closer to CPU)
- Many students saying spatial locality is moving contiguous block of data about in the hierarchy rather than highlighting the idea of likely accessing neighbouring data to a used data point
- Explanations in general: varied quality, not many good ones but that seems to be mostly due to poor writing rather than lack of understanding (iff they get the definitions correct)
- A number of students talk about instructions rather than data. If explanations are roughly correct 0.5 is given.
- Lots of students had the hierarchy inverted (calling lower cache levels higher)
- Lots refer to reducing "search" time for locations

Question B3 [8 marks]

Consider that an InfraRed Module (IRM) is hooked up to three GPIO pins on an ARM processor. The diagram below shows the block diagram of the IRM module. The module has a somewhat non-standard interface. It has the following digital signal lines and operation is explained below:

IR_DOTX : input signal to IRM to request it to sent bit on DATX line over IR
IR_DORX : input signal to IRM to request it to return latest received bit on DARX line
IR_DATX : input signal to IRM to indicate data bit to send over IR
IR_DARX : output signal from IRM to indicate latest data bit received over IR
IR_RDY : output signal from IRM to say it is ready (bit sent or bit received)

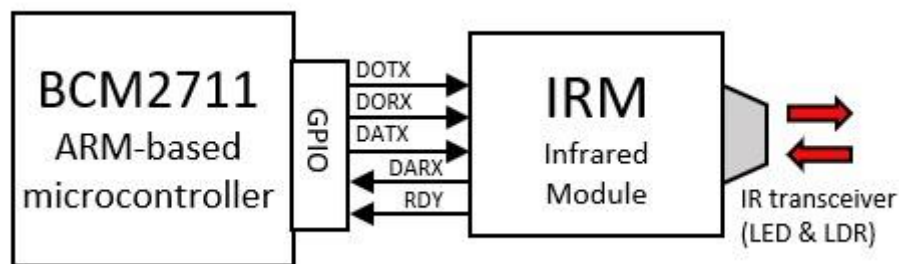


Figure showing ARM microprocessor to IRM connections

Note that IRM responds onto to positive edges on the IR_DOTX and IR_DORX lines; whenever it receives a positive edge on either of these lines it will immediately lower the IR_RDY line, and then only raise that line wither once it has sent a bit or receives a bit over IR. To send a bit, the microprocessor must wait for RDY to be high. Then, the DATX must first be set with the bit to send and then only can DOTX be raised to tell the module to transmit the bit on DATX over IR.

The IRM module is reset by setting both IR_DOTX and IR_DORX to high (it will only exit reset mode once either of these lines have been lowered, this is already done in the baseline code).

Have a look at the code below that configures the GPIO and resets the IRM. It coincidentally also indicates a piece of code that is missing, that you are asked to think about in questions B3.1 and B3.2 below.

```
/* Program to send the binary sequence 1010 over IR and then wait until a reply
   is sensed. */

#include <stdio.h>
#include <wiringPi.h>

/* device connected to pins 0 to 4 on BCM_GPIO 17 */
#define IR_DOTX    0 // request IRM to sent bit on DATX line over IR
#define IR_DORX    1 // request IRM to return latest received bit on DARX line
#define IR_DATX    2 // tell IRM what bit to send over IR
#define IR_DARX    3 // indicates what data bit IRM received over IR
#define IR_RDY     4 // if pin high then IRM is ready to receive command signals
                   // if pin low then is is busy or waiting for tx/rx to complete

int main (void)
{
    printf ("Welcome to IRM tester\n") ;

    // set up GPIO to connect to IRM
    wiringPiSetup ();
    pinMode (IR_DOTX, OUTPUT);
    pinMode (IR_DORX, OUTPUT);
    pinMode (IR_DATX, OUTPUT);
    pinMode (IR_DARX, INPUT);
    pinMode (IR_RDY, INPUT);
```

```

// reset IRM by raising both DOTX and DORX
digitalWrite (IR_DOTX, 1);
digitalWrite (IR_DORX, 1);
// lower both DOTX and DORX so that IRM is running
digitalWrite (IR_DOTX, 0);
digitalWrite (IR_DORX, 0);

while (1)
{
    // send the binary sequence 1010 over the IRM

    // >>>> Question B3.1 CODE TO BE ADDED HERE!!!! <<<<

    // wait for 500ms
    delay (500) ;
    // now see if anything has come back, if so write SUCCESS otherwise try again ...
    // BUT is this code good enough to do so? -- see Question B3.2
    if (digitalRead(IR_DARX)==1) {
        printf("SUCCESS!\n");
        return 0;
    }
    // nothing got returned so just delay a moment and try sending again
    delay (500) ;
}
return 0 ;
}

```

B3.1. The code provided above is not complete. Fill in the code, to be placed after the comment “Question B3.1 CODE TO BE ADDED ”, to make the microprocessor tell the IRM module to send out over IR the sequence of bits 1, 0, 1, 0. (You need to provide some code in your solution, not necessarily all of it; your answer can incorporate some code and some explanation of how you would solve this).

[4]

ANSWER: the student can either write some code showing a solution, or explaining the operation of the IRM interface, needing to wait for it to be RDY. Then putting the relevant bit on DATX. Then raise DOTX and lower it shortly after (or wait for RDY to transition from low to high). Repeat this for the other bits to send. Student could either do a for loop (after constricting e.g. an array of bits to send) or could explicitly send one after the another (duplicating a whole lot of code). An inefficient but function solution could get say 3/4. A good efficient solution 4/4. e.g.

```

static int ary[4] = {1,0,1,0};
for (int i=0; i<4; i++) {
    while (digitalRead(IR_RDY)==0); // wait for module to become ready
    digitalWrite (IR_DATX, ary[i]);
    digitalWrite (IR_DOTX, 1); // assumes DOTX already low, i.e. posedge
    digitalWrite (IR_DOTX, 0);
}

```

B3.2. Look at the code above, where it says “*BUT is this code good enough to do so?*”. At that point in the program, the program is essentially waiting to see if the IRM module has received an incoming IR message and has a bit available to be received. Discuss whether or not the code provided will achieve this objective, if you think so briefly explain why, or if you don’t think it would work explain how you might fix it. (Note: You do not have to provide code in your answer, a written explanation is sufficient).

[3]

ANSWER: The problem with the existing code is that, for starters, it has not checked to see if the IRM is ready to receive any command signals. So a loop may be needed first, before checking on received data, to wait for the IRM’s IR_RDY to become high. Then, before checking IR_DARX, it first needs to send a positive edge on IR_DORX (i.e., assuming IT_DOTX is low, it can do a `digitalWrite (IR_DORX, 1);` and then bring it back to a low setting with `digitalWrite (IR_DORX, 0);`). Then it needs to wait to see if IR_RDY to become high, having it check `digitalRead(IR_DARX)==1` is actually only going to check if a 1 data bit was received over IR, that was not specifically asked for, just to see if data (a 0 or 1) was received, so it should be after IR_RDY changing to 1 that is the point it can decide success. The code would be further improved by having a loop and timeout to check IR_RDY and what by was received.

B3.3. If you were not using WiringPi for using GPIO suggest an alternate approach or library that you could make use of instead.

[1]

ANSWER: Could develop a custom device driver for the device, e.g. a kernel module that can either be loaded when needed or included into the Raspberrian Linux kernel. Could use C pointers, or assembly, to access the memory addressed peripherals if running in system mode, or could read/write the files inside `/sys/class/gpio`

SECTION C:

TESTING YOUR METTLE QUESTIONS [34 marks]

Note: All questions in this section are compulsory. This section provides an additional opportunity to pass GA5 aspects B.1, A.3, B.5, and B.6.

The Following Scenario Relates to Questions C.1 - C.4 that follow

Take some time to read through this aspect briefly; the time for the exam is planned on you taking about 15 minutes reading and making sense of the scenario, and then answer the questions that follow.

Scenario: Configurable Wind-Powered Monitor (CWPM) IoT Platform

Consider that you are involved in developing an embedded platform system, called the Configurable Wind-Powered Monitor (CWPM) platform. The CWPM connects to a mini wind turbine that generates power, and charges a battery for times when the wind is not blowing. The CWPM design is shown in Figure 1 on the next page.

As indicated, the rotor connects via the axle to the generator. Different size rotors can be used, from small to large, and the generator part has automatic gearing to account for changes in torque and speed depending on the rotor in use (but no need to worry about that at the moment). The CWPM comes standard with a low power ARM-based microcontroller. There are two SPI expansion slots to connect up daughter boards (e.g. for additional transducers or sampling sensors). There is also an 4-channel 10-bit ADC and a single channel 14-bit DAC. The ADC channels ch2 to ch4 each connect to an input socket that can link to an analog sensor (that is restricted to signals in the range 0V to 5V). The first ADC channel, ch1, is used to sample the battery charge level (which will also return an analogue voltage from 0V to 5V), to see how much battery power is left. The dynamo has a switch that triggers on each rotation (i.e. if it is generating some current), each rotation sends a pulse on the rota_IRQ line that invokes an IRQ on the ARM processor.

Baseline application code that has already been developed is shown on the next two pages, after which the questions for this section follow on page 12.

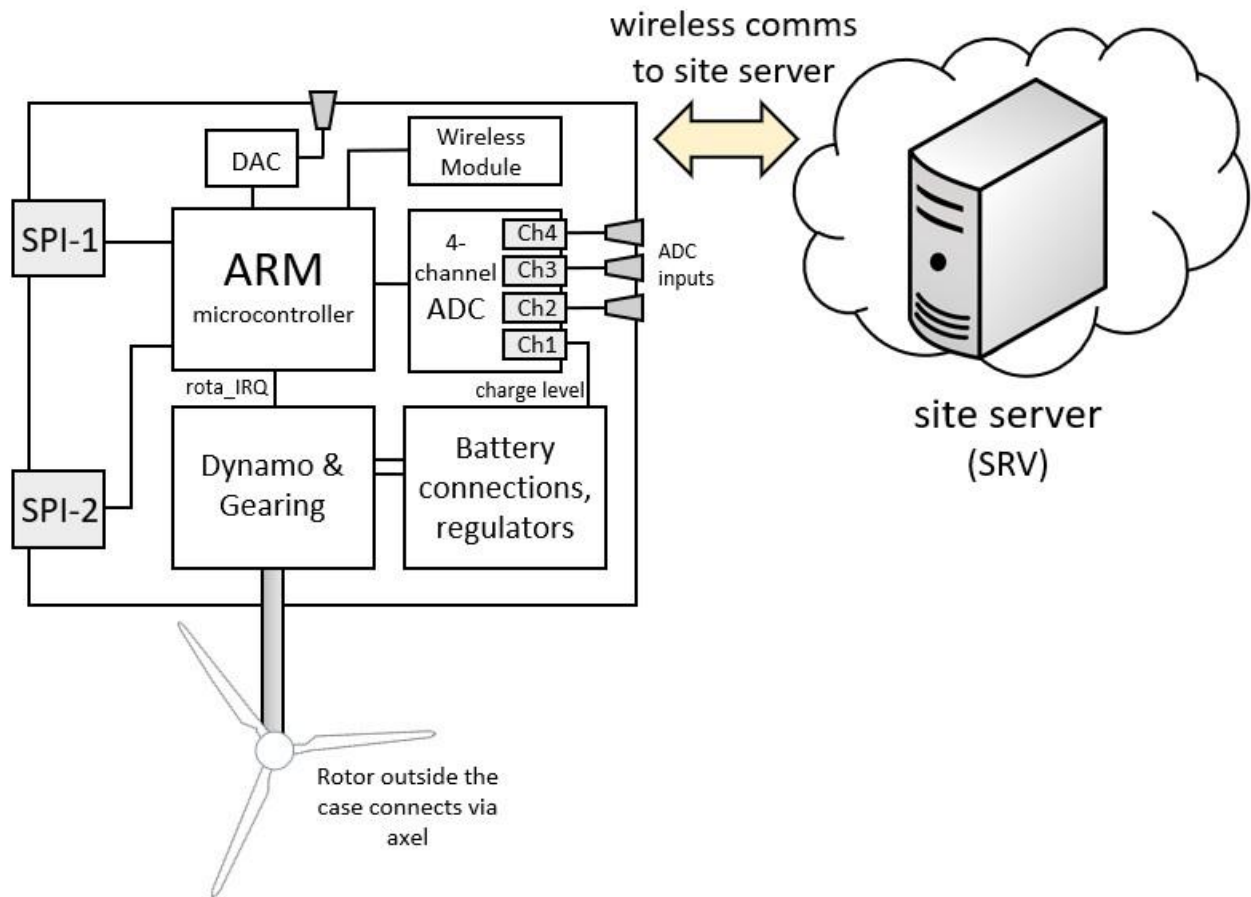


Fig. 1: Illustration of Configurable Wind-Power Monitor (CWPM) IoT Product

Data Capture and Buffering

A significant design decision for the CWPM was that it would gather input from its various sensors and store these in an array of structures for each sensor. These structs would essentially be used as a FIFO, new data will be inserted to the struct and data will be pulled out from the tail when ready to transmit. The struct would have the following form:

```

/** structure to store time information */
typedef struct {
    unsigned char day, month, year; /* year is offset from 2000 */
    unsigned char hour, minute, second;
} Time;
/** structure to store temperature sample */
typedef struct {
    Time tstamp; /* date and time */
    float tempC; /* temperature in degrees C */
} TempSensor;

/* FIFO buffer used to store temperature data */
#define FIFO_LEN 0x100 /* specify number items in FIFO */
TempSensor temps[FIFO_LEN]; /* recorded temperature samples */
int head=0, tail=0, ntemps=0; /* FIFO head and tail, and count of samples */

```

Functions available for timing

The following functions have already been defined elsewhere in the program:

```
/* Externally defined functions */
extern unsigned long getticks_us(); /* time from system startup in microsec
*/
extern void delay_ms(int ms); /* Cross-platform millisecond sleep function
*/
void getTime( Time* t ); /* incomplete function */
```

Sending data to the server

The sendall function below is invoked in report mode when the IoT device has gathered enough data. The implementation of this function is shown below (and relates to some questions that follow).

```
void sendall () {
    while (ntemps>0) {
        tail = (tail + 1)%LEN;
        ntemps--;
        wsend_data(MSG_TEMP,&temps[tail],sizeof(temps[tail]));
    }
}
```

Baseline main function

The following starting point main() function has been put together:

```
/** Main module: entry point to CWPM baseline */
void main () {
    float tempC; /* temporary variable to store temperature */
    printf("CWPM client started!\n");
    /* baseline just collects temperature and sends it to site server */
    while (1) {
        tempC = tempsens_read(); /* get the temperature in deg C */
        if (ntemps<FIFO_LEN) {
            temps[head].tempC = tempC; /* add to FIFO */
            getTime(&temps[head].tstamp); /* get RTC */
            head = (head + 1)%FIFO_LEN; /* move FIFO head on */
            ntemps++; /* another sample added */
        }
        /* wait 1s and then check if need to send all samples */
        delay_ms(1000);
        if (ntemps>204) sendall();
    }
}
```

Answer the following questions on the question paper in the space provided below the question...

Question C.1 [9 marks]

This question provides a reattempt at GA5-B.1 Embedded System design as a complex process.

The CWPM device involves an ARM microprocessor that connects to memory mapped peripherals. The plan is to provide C code, and GCC, for using the devices on the platform. Answer the following questions:

C.1.1. Consider that you have been tasked with deciding the Development Environment that engineers working on the embedded software of the CWPM will be utilizing¹. Explain what is meant by the development environment (1 mark) and explain why a development environment for a custom embedded system may be different to that used for building application software for a standardized PC. (2 marks).

[3]

ANSWER: The development environment was explained in lectures as the work space that development of the embedded system takes place. e.g. lab environment. But more than that, it includes both equipment, hardware and software needed for the development. Equipment such as logic analyser and signal generator; hardware needed such as peripherals that may be hooked up to the embedded platform, the embedded platform of course, development PC, needed software installed on the development PC etc.

C.1.2. The CWPM has the *rota_IRQ* line hooked up to the IRQ line of the ARM processor. But there may likely be a need for other peripherals, such as those that connect to daughterboards, to provide interrupt signals. Discuss how you could work around this problem, why it could be possible, even if hardware changes to the platform are needed, so that the ARM can handle more than one interrupt.

[3]

ANSWER: For one thing, the ARM has two interrupt lines, the FIQ and the IRQ. The FIQ could be wired up to handle other interrupts that it may need to detect. But that is not a very elegant solution, considering that the *rota_IRQ* is not necessarily going to be so high priority; sure it would be nice to know if power is being generated, but that interrupt IRQ line could be shared with other peripherals. Additional hardware would need to be added then, such as an advanced interrupt controller (AIC) if the microprocessor does not already have that facility in place already.

¹ You can assume all the engineers in the team have C programming and other ES development skills.

C.1.3. In the design, there is a likely need to call manually written assembly code functions from C functions. Briefly explain how a C function could be connected to an assembly function. (While you can score full marks explaining ARM C ABI approach, you can still get marks for suggesting a potentially effective approach for making such a connection. Assembly code snippets can be added to aid explanation).

[3]

ANSWER: If using the ARM C ABI, there are various alternatives used in terms of stack usage for saving registers that are utilized in the function being involved. These are:

- Store Before Call (SBC)

OR

- Call Before Store (CBS) also sometimes named Store Before Use (SBU)

(possibly less optimal than above but more reliable)

When utilizing both options, the stack may be utilized to restore the value of registers needed by the calling function to the state they were in when the called function returns.

The standard procedure by the ABI is that :

The first 13 input parameters are passed to the function using registers r0 to r12 (any additional values are passed using the stack)

The return value of the function (if it returns anything) is returned using r0.

Question C.2 [8 marks]

This question provides a reattempt at GA5-A.3 analysis of embedded software designs and performance.

C.2.1. The **Data Capture and Buffering** section above indicates the struct and FIFO array used to store up to FIFO_LEN samples from the temperature sensor. To send the data from the CWPM to the site server, the *sendall* function shown above is used. The amount of data sent seems quite wasteful considering the information transferred. Such as, the temperature is sent as a float (even though it ranges from -49.9°C to 120.9°C accurate only to 1/10 of a degree). Suggest an approach to cut down on the amount of data being sent. (explain the amount of data if any your suggestion might save).

[3]

ANSWER: A standard float (on a 32-bit architecture) uses 32-bits. That is 4 bytes of data for sending the temperature. However the range of values that the temperature sensor value can take on ranges from -499 to 1209. That is 1709 values (including 0). This can easily fit into a 16-bit, ie. 2-byte value (it can even fit into 12-bits, so 4 bits might still be zero, those could be used for other things, e.g. parity checking). Using a 16-bit short int for temperature will save 2 bytes per packet.

Much more data saving could be done by having occasional time check synchronization packets (to synch the IoT node with the server) and then the node can just send an offset from the time check (in seconds) instead of sending the entire Time structure. If the offset was say also 16-bits (2 byte number) then the data saved per packet would be `sizeof(TempSensor) - 4`. TempSensor has 10 bytes. This, this optimization would utilize only 4/10 (i.e. 40%) of the previous data usage, potential comms speed up to 60% improved.

C.2.2. The CWPM has the *rota_IRQ* line that drives the IRQ request line on the ARM. The starting point, in C, for this routine is shown below.

What is needed is a means to track how fast the dynamo is turning, and we want to **determine an averaged speed of rotation over a period of 120s**. If the *rota_event* ISR function is called for each rotation of the dynamo, explain how you could go about working out the speed in RPM (i.e. rotations per minute) that the dynamo is turning. Keep in mind the functions, mentioned earlier, that are available: *getticks_us*, *delay_ms* and *getTime*. You want to try keeping the code in the ISR minimal. You can describe or annotate in the code below how you might change things in order to achieve this requirement..

[5]

```
/** CWPM interrupt service
    The __irq declaration ensures that the return from the function
    actually implements a return from interrupt changing back to
    the prior processor mode. You can access global variables as
    normal in the interrupt routine. */

void __irq rota_event () {

    // currently does nothing just returns

}

/** Main function would be here, it is shown on page XX */
void main ()
{
    ... existing code ...
}
```

ANSWER: Can simply use three global variables:

- one to save the last timestamp such as unsigned long *t_pre*;
- An unsigned *n_rotations* to keep track of the number of rotations, initialized to 0).
- A sum, *sum_rotations*, to keep a running total of the sum of rotation times setting

Then in the interrupt a local var could be set to *t_rotated* = *getticks_us*() - *t_pre*; to see how long it took between rotations and then add this to *sum_rotations* and increment *n_rotations*.

This can all be done fairly efficiently in the interrupt. Alternatively, the interrupt could just get a timestamp and indicate (e.g. using counter) that a new timestamp has been received (that would be about two instructions as being accessing global memory that would probably be in RAM/cache). The main function can then check and see if the count changes and update the average accordingly. Averaging over 120s would add complications, where one needs to keep track of when 120s has been reached and e.g. calculate the average from the sum at that point and display / send that value to the server.

{comment to external: perhaps this should be worth more than 4 marks?! I could take out the 120s average although it's not difficult to compute that.}

Question C.3 [8 marks]

This question provides a reattempt at GA5-B.5, using (cross-)compiler for developing ES software.

C.3.1. GCC will be used for the C software needed for the CWPM. Installing IDEs, GCC, and related libraries would put much demand on the low-power processor and its limited memory. Accordingly, use of cross-compiler tools are considered. Briefly explain what advantages would be obtained from using a cross-compiler for this project. What sort of configuration might be needed, what would you need and do to get compiled code to run on the target (i.e. CWPM) platform?

[4]

ANSWER: Cross compiler could run faster on development PC as opposed to running it on the embedded platform. Installing the compiler and IDE, and other tools needed for development could take up a significant amount of space on the embedded platform which could pose problems in fitting application code, libraries and possibly other resources and custom software that may be needed. Furthermore, if different processors and platforms are used in the project, having cross-compilers set up for the various needed platforms on one machine could make the development process more efficient (as in being able to quickly swap compilers or compile the same code modules for different architectures in one make file).

C.3.2. There are various coding strategies that can improve the efficiency of code. One of these is the process or loop unrolling. Briefly explain what is meant by the process of loop unrolling and provide a brief example showing what sort of saving can be achieved (for your example you could formulate a rough estimate in terms of the saving, if any, provided).

[4]

ANSWER: Loop unrolling is an optimization strategy whereby one iteration of a loop is expanded out to do multiple interactions (e.g. the counter stepping every second value instead of every value, and the iteration of the loop working on two values possibly at the same time instead of just one value). This is a form of space-time tradeoff, whereby more code may end up being procedure but it can be done faster e.g. using parallel processing. In some cases the compiler may be able to work out iterations that can even be skipped or substituted (e.g. if three iterations just end up adding 1 to a sum, then those iterations could perhaps just be compressed to one iteration that adds 3 to the sum instead, obviously a simple case like: `int sum = 0; for (int i=0; i<10; i++) sum++;` could be unrolled to simply. `sum+=10;`

Question C.4 [8 marks]

This question provides a reattempt at GA-B.6 standard embedded systems communication protocols

The CWPM is designed around supporting additional plug-on daughter boards that will be connected to via a SPI port. The SPI port interface has four signals: SCLK, MOSI, MISO, and SS.

C.4.1. Let's consider that we want to send over SPI the byte 0x5B from the master to the daughterboard (an SPI slave). Assume lines are in idle state, that neither master or slaves are communicating.. For your answer, you can either show a diagram of how the signals may look or you can explain in text/code how the communication will work to send this byte to the slave (assume the slave successfully receives the data sent).

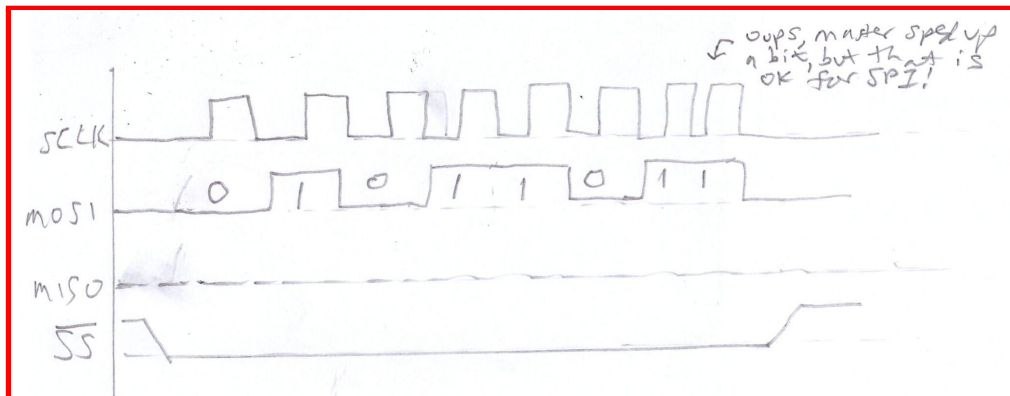
[5 marks]

ANSWER: The student would first need to convert the byte to binary so as to know which bits are going to be sent serially, using the MOSI line for the most part since it is sending from Master to Slave.

0x5B = 01011011 (the MSB is sent first)

The SCLK needs to keep going for each MOSI and the !SS line needs to be low during this time, indeed going low when the first MOSI is going to be set. The SCLK only goes high after the MOSI is set.

The diagram below shows the waveforms:



del figure for question paper!!!

C.4.2. The I2C protocol has a number of advantages, but also some disadvantages compared to SPI. Briefly discuss one advantage of I2C over SPI, and one advantage that of SPI has over I2C.

[3 marks]

ANSWER:

I2C or I²C = Inter-Integrated Circuit SPI = Serial Peripheral Interface

Major advantage of I2C is it is a bit simpler in terms of interface and lines used: I2C is a 2-wire bus protocol, with only the lines SDA (Serial Data) and SCK (Serial Clock). SPI, in contrast, needs four wires to control just a single slave, i.e. SCK, MOSI, MISO, and a slave select (SS) line (although if one only ever uses it for one master to one slave, and you don't mind the slave being always busy listening to SCLK, then SS could be forced to 0 (saving a wire, but that's not really recommendable and not how the protocol is really meant to be used).

SPI has further benefits: SPI is full-duplex communication, whereas I2C can only be half-duplex.

I2C is limited to around 400Kbps usually, ultra fast versions up to 5Mbps (but not so common). SPI does not really have a defined speed limit, it is designed more to cater for high speeds, can go up to 10Mbps, some implementations even faster.

- I2C (since it's bidirectional) is not easy to isolate such as with an opto-isolator, and so cunning electronic

design is needed whereas SPI is quite straight forward to isolate.

SECTION D:

TESTING YOUR METTLE QUESTIONS [24 marks]

Each of the following questions is a GA testing question iff you are 1 of the few students granted provisional DP due to lack of GA testing opportunities following illness.

Question D.1 Finite State Machines [8 marks]

This question tests GA B.2

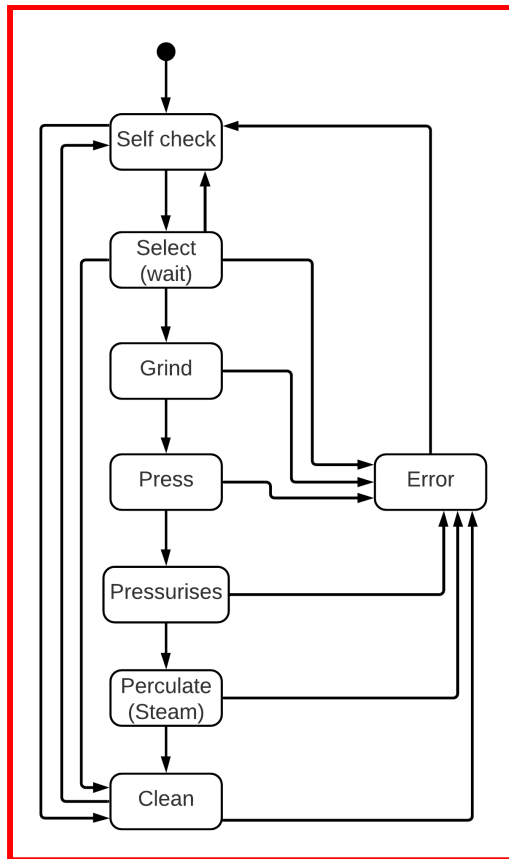
D1.1 You're writing the code for the microprocessor used to control a new beans-to-cup coffee machine. Draw the FSM that would most efficiently implement the following system functionality.

Making coffee requires a user to select to make a cup of coffee. The machine will then grind the beans, press the ground beans into the steam cup, pressurise and heat the water, and then force the water through the grounds (percolate the coffee) into the cup, and then perform a self clean. This machine does a series of self checks whenever it's turned on and after it self cleans (checks water level, bean level, if it's grounds dump container is full or not, how long ago the last clean was done, and if there are ground beans in the steam cup). If any of the self checks indicate a user needs to do something (refill the water, refill the beans, empty the dump container), then it will indicate the required action in a touch screen and wait for the user to do the action, once done it will then continue to wait for the user to select to make coffee. The self check also looks at how recently it last did a clean and if more than 12hrs has past it will do a clean. At any point in time, if an error occurs it will do a self check and once the error is resolved it will return to waiting for a user to select to make coffee.



Do not show inputs and outputs on the FSM diagram.

[4]



delete this answer fig for q paper!

D1.2 How do mealy and moore FSMs differ? (mention at least four points)

[4]

ANSWER: any 4 points:

Mealy:

- Outputs depend on both current state and inputs
- Asynchronous
- Requires fewer states

Moore:

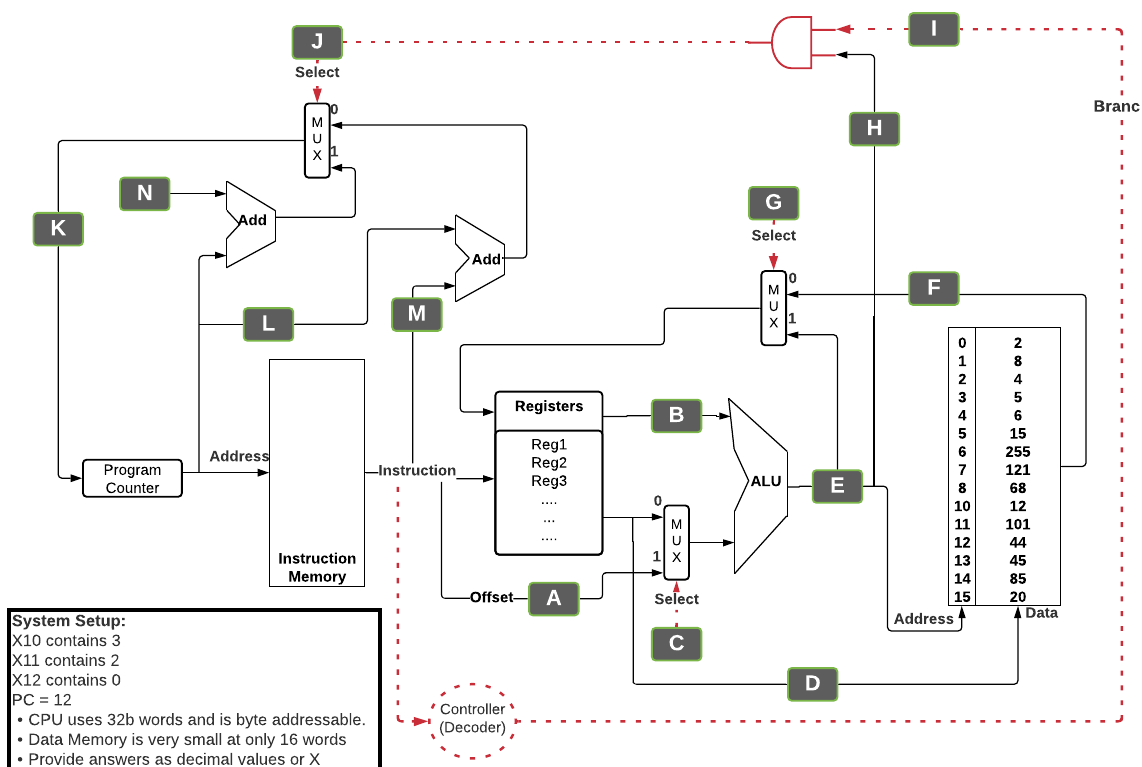
- Outputs depend on current states only
- Synchronous
- Requires more states

Question D.2 Computer and microprocessor architecture [8 marks]

This question tests GA A.2

D.2.1 Fill in the lettered boxes (see next page) if the following instruction was to be executed:

LDR X10, [X11, 4]



[4 marks]

Fill in the values for the signals in the boxes below:

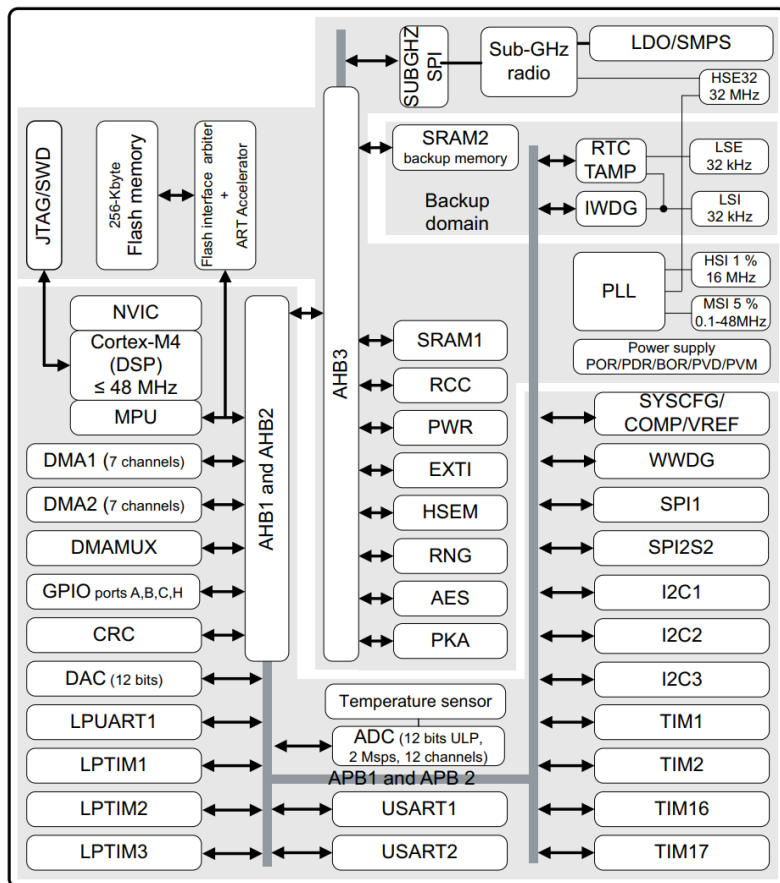
A		H	
B		I	
C		J	
D		K	
E		L	
F		M	
G		N	

ANSWER:

A	4	H	x/0
B	2	I	0
C	1	J	1
D	x	K	16
E	6	L	x/12
F	255	M	x
G	0	N	4

D.2.2 The figure below shows the core functional components in a STM32WLE5. This microprocessor includes 2 CPU cores, a LoRa radio, and multiple interfaces.

Figure 1. STM32WLE5/E4xx block diagram



- a) Give an example of of an application that this shown IC would be appropriate for and explain why: [2 marks]

ANSWER: Any reasonable application and explanation: Eg expect:

- Something that needs lora connectivity
- Something that needs a small footprint (lots packed into a single IC)
- Something that needs DSP processing (eg Audio)

- b) The IC diagram shows 2 SRAM units and a portion of flash memory. Why would both flash and SRAM memory be required? List 2 reasons. [2 marks]

- Flash is non-volatile/larger
- SRAM is faster than Flash

Question D.3 Verilog [8 marks]

This question tests GA A.1

All questions refer to the following code snippet

```
1  `timescale 1ns / 1ps
2
3  module exam (input A, B, clk, output Y, Z);
4      reg Z=1'b0;
5
6      assign Y = A & B;
7
8      always @(posedge clk) begin
9          //always @(A) begin
10             Z <= A & B;
11         end
```

D.3.1 Given the above code as it is, list 3 differences between Y and Z?

[3]

ANSWER:

- Y will update as soon as A or B change
- Z will only update on the rising clock edge
- Z will start at 0 while Y will always just be whatever A&B drives it to be.

D.3.2 If line 8 is commented out instead and line 9 uncommented what would change about the synthesized logic's behaviour? [2]

ANSWER:

Z won't change even if B changes, and Z will change as soon as A changes not only on rising clock edges

D.3.3 Write the 1 line of verilog required to instantiate the 'exam module' as if in a testbench.

[2]

ANSWER:

`exam ex1(A, B, clk, Y, Z);`

D.3.4 Write down the code that will correct the 1 coding error in the above module implementation.

[1]

ANSWER: `endmodule`

END OF EXAMINATION

Appendix A: Verilog Cheat sheet

Numbers and constants

Example: 4-bit constant 10 in binary, hex and in decimal: `4'b1010 == 4'ha -- 4'd10`

(numbers are unsigned by default)

Concatenation of bits using {}

`4'b1011 == {2'b10, 2'b11}`

Constants are declared using parameter:

`parameter myparam = 51`

Operators

Arithmetic: and (+), subtract (-), multiply (*), divide (/) and modulus (%) all provided.

Shift: left (<<), shift right (>>)

Relational ops: equal (==), not-equal (!=), less-than (<), less-than or equal (<=), greater-than (>), greater-than or equal (>=).

Bitwise ops: and (&), or (|), xor (^), not (~)

Logical operators: and (&&) or (||) not (!) note that these work as in C, e.g. `(2 && 1) == 1`

Bit reduction operators: [n] n=bit to extract

Conditional operator: ? to multiplex result

Example: `(a==1)? funcif1 : funcif0`

The above is equivalent to:

`((a==1) && funcif1)`

`|| ((a!=1) && funcif0)`

Registers and wires

Declaring a 4 bit wire with index starting at 0:

`wire [3:0] w;`

Declaring an 8 bit register:

`reg [7:0] r;`

Declaring a 32 element memory 8 bits wide:

`reg [7:0] mem [0:31]`

Bit extract example:

`r[5:2]` returns 4 bits between pos 2 to 5 inclusive

Assignment

Assignment to wires uses the assign primitive outside an always block, e.g.: `assign mywire = a & b`

Registers are assigned to inside an always block which specifies where the clock comes from, e.g.:

`always@(posedge myclock)`

`cnt = cnt + 1;`

Blocking vs. unblocking assignment <= vs. =

The <= assignment operator is non-blocking (i.e. if use in an `always@(posedge)` it will be performed on every positive edge. If you have many non-blocking assignments they will all be updated in parallel. The <= operator must be used inside an always block – you can't use it in an assign statement.

The blocking assignment operator = can be used in either an assign block or an always block. But it causes assignments to be performed in sequential order. This tends to result in slower circuits, so

avoid using it (especially for synthesized circuits)
unless you have to.

Case and if statements

Case and if statements are used inside an always block to conditionally update state. e.g.:

```
always @(posedge clock)
  if (add1 && add2) r <= r+3;
  else if (add2) r <= r+2;
  else if (add1) r <= r+1;
```

Note that we don't need to specify what happens when add1 and add2 are both false since the default behavior is that r will not be updated. Equivalent function using a case statement:

```
always @(posedge clock)
  case({add2,add1})
    2'b11 : r <= r+3;
    2'b10 : r <= r+2;
    2'b01 : r <= r+1;
    default: r <= r;
  endcase
```

Module declarations

Modules pass inputs, outputs as wires by default.

```
module ModName (
  output reg [3:0] result, // register output
  input [1:0] bitsin, input clk, inout bidirectnl );
... code ...
endmodule
```

Verilog Simulation / ISIM commands

```
$display ("a string to display");
$monitor ("like printf. Vals: %d %b", decv,bitv);
#100 // wait 100ns or simulation moments
$finish // end simulation
```

Appendix B: ARM Assembly Language Cheatsheet

Memory access instructions

```
LDR Rd, [Rn]           ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]      ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]           ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]      ; store 32-bit Rt to [Rn+off]
PUSH {Rt}              ; push 32-bit Rt onto stack
POP {Rd}               ; pop 32-bit number from stack into Rd

MOV{S} Rd, <op2>       ; set Rd equal to op2
MOV Rd, #iml6           ; set Rd equal to iml6, iml6 is 0 to 65535
MVN{S} Rd, <op2>       ; set Rd equal to -op2
```

Branch instructions

```
B label                ; branch to label Always
BEQ label              ; branch if Z == 1 Equal
BNE label              ; branch if Z == 0 Not equal
BCS label              ; branch if C == 1 Higher or same, unsigned ≥
BHS label              ; branch if C == 1 Higher or same, unsigned ≥
BCC label              ; branch if C == 0 Lower, unsigned <
BLO label              ; branch if C == 0 Lower, unsigned <
BMI label              ; branch if N == 1 Negative
BPL label              ; branch if N == 0 Positive or zero
BVS label              ; branch if V == 1 Overflow
BVC label              ; branch if V == 0 No overflow
BHI label              ; branch if C==1 and Z==0 Higher, unsigned >
BLS label              ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label              ; branch if N == V Greater than or equal, signed ≥
BLT label              ; branch if N != V Less than, signed <
BGT label              ; branch if Z==0 and N==V Greater than, signed >
BLE label              ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                  ; branch indirect to location specified by Rm
BL label               ; branch to subroutine at label
BLX Rm                 ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, <op2> ; Rd=Rn&op2 (op2 is 32 bits)
ORR{S} {Rd}, Rn, <op2> ; Rd=Rn|op2 (op2 is 32 bits)
EOR{S} {Rd}, Rn, <op2> ; Rd=Rn^op2 (op2 is 32 bits)
BIC{S} {Rd}, Rn, <op2> ; Rd=Rn&(~op2) (op2 is 32 bits)
ORN{S} {Rd}, Rn, <op2> ; Rd=Rn|(~op2) (op2 is 32 bits)
LSR{S} Rd, Rm, Rs      ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n      ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs      ; arithmetic shift right Rd=Rm>>Rs (signed)

ASR{S} Rd, Rm, #n      ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs      ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n      ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, <op2> ; Rd = Rn + op2
ADD{S} {Rd}, Rn, #iml2 ; Rd = Rn + iml2, iml2 is 0 to 4095
SUB{S} {Rd}, Rn, <op2> ; Rd = Rn - op2
SUB{S} {Rd}, Rn, #iml2 ; Rd = Rn - iml2, iml2 is 0 to 4095
RSB{S} {Rd}, Rn, <op2> ; Rd = op2 - Rn
RSB{S} {Rd}, Rn, #iml2 ; Rd = iml2 - Rn
CMP Rn, <op2>          ; Rn - op2 sets the NZVC bits
MUL{S} {Rd}, Rn, Rm     ; Rd = Rn * Rm signed or unsigned
```


Notes

Ra Rd Rm Rn Rt represent 32-bit registers

value any 32-bit value: signed, unsigned, or address

{S} if S is present, instruction will set condition codes

#im12 any value from 0 to 4095

#im16 any value from 0 to 65535

{Rd,} if Rd is present Rd is destination, otherwise Rn

#n any value from 0 to 31

#off any value from -255 to 4095

label any address within the ROM of the microcontroller

op2 the value generated by <op2>